Бекенд без паніки

Три проєкти: basics, core, patterns. Що важливо, коли важливо, а коли можна забити.

Це не довідник і не курс. Це відповідь на одне питання для кожної теми: «навіщо мені це і коли?». Код, тести і подробиці лежать у репозиторіях, тут тільки те, що треба мати в голові, щоб не читати все підряд.

Порядок

- **basics** (github.com/vladio71/backend-1-basics): один вечір. Зрозуміти чотири шари. Розділ 1 тут про них.
- **core** (backend-2-core): основне. По одній темі за раз, починати з learn/ на localhost:5173: там кожна тема з живими кнопками. Розділ 3 тут.
- **patterns** (backend-3-patterns): коли core уже не лякає. Розділ 4.

Як читати картки

Кожна тема в розділах 3 і 4 це одна картка з шістьма полями. Розділ 2 показує таку картку на прикладі модуля з LLM, щоб було видно, як нею користуватись.

Що це	одним абзацом, без термінів, де можна
Чому важливо	що зламається, якщо цього не буде
Коли про це думати	на якому етапі проєкту це треба закласти
Коли можна забити	чесно: в яких проєктах це зайве
При якому трафіку	орієнтир у цифрах, не закон
Як зрозуміти, що час настав	конкретний сигнал із логів, метрик або скарг

Цифри трафіку в картках це орієнтири з практики, а не закон. Вони кажуть, з якого порядку величин тема перестає бути теорією. Коли сумніваєшся, подивись на метрики: розділ 5 каже, де вони.

1. Чотири шари. Чому код розкладено саме так

Уяви кав'ярню. Бариста за стійкою приймає замовлення і віддає каву, але сам нічого не вирішує. Менеджер вирішує: чи є зерно, чи можна в борг, що робити, якщо клієнт просить неможливе. Комірник ходить на склад і приносить те, що попросили. Склад просто тримає речі на полицях. Бекенд влаштований так само, і кожна роль живе в окремому файлі.

Контролер (бариста)	loans.controller.ts
Робить	Приймає HTTP-запит і віддає відповідь. Знає тільки: на POST /loans треба покликати сервіс.
Типова помилка	Пхати сюди логіку: перевірки, підрахунки, запити до бази. Через місяць контролер на 400 рядків, і ніхто не знає, де що.
Як перевірити себе	У контролері немає жодного if, крім перевірки прав. Кожен метод це 1-3 рядки: взяти dto, покликати сервіс, повернути.

Сервіс (менеджер)	loans.service.ts
Робить	Усі рішення: книги немає, значить 404. Примірників нуль, значить 409. Інакше видаємо.
Типова помилка	Писати SQL прямо тут або повертати HTTP-відповіді. Сервіс не має знати, що існує HTTP. Тоді його можна викликати з воркера чи з тесту.
Як перевірити себе	Сервіс можна протестувати без бази і без HTTP, підставивши фейковий репозиторій. У basics так і зроблено.

Репозиторій (комірник)	loans.repository.ts
Робить	Тільки читає й пише в базу. Одна транзакція: списати примірник і записати видачу разом.
Типова помилка	Приймати рішення: «якщо примірників нуль, кинути помилку». Це робота сервісу. Репозиторій тупий, і це добре.
Як перевірити себе	Кожен метод репозиторію це один запит або одна транзакція. Жодних бізнес-слів у назвах, тільки findById, save, lock.

Сутність (полиця)	loan.entity.ts
Робить	Опис таблиці: які поля, якого типу, що з чим пов'язане. Міграція створює таблицю за цим описом.
Типова помилка	Класти в сутність методи з логікою або ходити з неї в базу. Сутність це структура даних і нічого більше.
Як перевірити себе	Сутність можна прочитати за 10 секунд і зрозуміти, як виглядає таблиця.

Швидкий тест, чи все на місці: візьми будь-яку функцію і спитай «це приймає, вирішує, ходить у базу чи описує таблицю?». Якщо відповідь «і те, і те», функція стоїть не там.

Той самий запит у коді basics

```
POST /loans { bookId: 3, readerName: "Олена" }
  → main.ts: ValidationPipe перевіряє тіло. Не те поле: 400, далі не йде
  → loans.controller.ts: @Post() borrow(dto) { return this.service.borrow(dto) }
  → loans.service.ts: книги немає: 404. примірників нуль: 409. інакше repository.borrow()
```

```
  → loans.repository.ts: одна транзакція: UPDATE books ... ; INSERT INTO loans ...
  → PostgreSQL: таблиці з book.entity.ts і loan.entity.ts
  ← 201 { id: 3, bookId: 3, readerName: "Олена", returnedAt: null }
```

Навіщо це фронтендеру. Ти вже робиш те саме: компонент не ходить у fetch сам, є шар API, є стор. Контролер це компонент, сервіс це стор із логікою, репозиторій це api-клієнт. Різниця тільки в тому, що помилка на бекенді коштує дорожче, тому дисципліна жорсткіша.

2. Зразок картки на живому прикладі

Беремо модуль ai з core і проганяємо його через шість питань. Так само читається кожна картка далі.

Зразок: LLM вирішує, пропускати пост чи ні (core, модуль ai)	core: modules/ai
Що це	Є ендпоінт POST /posts/:id/moderate. Бекенд бере текст поста, шле моделі промпт «схвалити чи відхилити, поясни чому» і просить відповісти строго у форматі { action: "approve" | "reject", reason: "..." }. Відповідь перевіряє zod: якщо модель повернула щось не те, пробуємо ще раз, потім 502. Рішення зберігаємо в пості.
Чому важливо	Без строгого формату модель відповідає як хоче: сьогодні JSON, завтра «Звісно! Ось моє рішення: ...». Код падає на парсингу. Без повтору і 502 один збій моделі перетворюється на помилку для користувача, а ти не розумієш, чия вона.
Коли про це думати	Коли вперше вставляєш LLM у бекенд. Контракт відповіді і що робити при смітті треба вирішити до першого рядка коду, а не після першого падіння в продакшні.
Коли можна забити	Коли LLM просто пише текст для людини (чернетка листа, опис товару) і його далі ніхто не парсить. Тоді строгий формат зайвий, досить таймауту.
При якому трафіку	Не про трафік, а про гроші й час: один виклик коштує 0.5-3 секунди і реальні центи. При 10 запитах на хвилину можна викликати прямо в запиті. При 100+ на хвилину модерацію треба винести в чергу, бо запит користувача не має чекати 3 секунди на модель.
Як зрозуміти, що час настав	У логах з'являються помилки парсингу відповіді моделі. p95 ендпоінту понад 2 секунди. Рахунок за API росте швидше за кількість користувачів. Будь-яке з трьох означає: формат, повтор і черга.

Шматок коду, про який уся картка

```
// Контракт відповіді. Усе, що не збігається, відкидається.
const Decision = z.object({
  action: z.enum(['approve', 'reject']),
  reason: z.string().min(3).max(300),
});

// Один повтор, потім чесна помилка нагору.
const raw = await llm.complete(prompt);
const parsed = Decision.safeParse(tryJson(raw));
if (!parsed.success) {
  const again = await llm.complete(prompt + '\nВідповідай тільки JSON.');
  const second = Decision.safeParse(tryJson(again));
  if (!second.success) throw new BadGatewayError('LLM_BAD_OUTPUT');
}
```

Що тут головне: не сам виклик моделі, а дві речі навколо нього. Контракт відповіді (zod) і чесна поведінка при збої (один повтор, потім 502 нагору, а не тихий null). Усе інше в модулі ai

це обв'язка.

3. core: основи, тема за темою

Шістнадцять карток у порядку навчання: спочатку те, без чого не напишеш жодного ендпоінту, потім те, що є в більшості проєктів, наприкінці спеціальне. Той самий порядок у README і в меню learn/.

Auth: JWT, refresh, ролі	core: auth
Що це	Після входу користувач отримує короткий пропуск (JWT на 15 хвилин) і довгий (refresh на 7 днів). Короткий кладе в кожен запит. Коли він протух, міняє довгий на нову пару. Старий довгий більше не працює; якщо хтось спробує його вдруге, усі входи цього користувача скидаються.
Чому важливо	Пропуск на рік без оновлення означає: вкрали один раз, користуються рік. Без ролей кожен ендпоінт сам вирішує, хто адмін, і десь забуде.
Коли про це думати	Коли з'являється перший ендпоінт, який не має бачити «будь-хто». Зазвичай другий день проєкту.
Коли можна забити	Внутрішній інструмент за VPN на 5 людей: можна один довгий токен. Але ротацію refresh усе одно краще зробити, це 40 рядків.
При якому трафіку	Не залежить від трафіку, залежить від ціни зламу. Є гроші або особисті дані, значить ротація обов'язкова.
Як зрозуміти, що час настав	Скарга «мене вилогінило» або, гірше, «хтось зайшов від мого імені». Друге вже пізно.

Валідація і формат помилок	core: errors, posts
Що це	Кожне тіло запиту перевіряється за DTO: зайве поле, не той тип, порожній рядок дають 400 з іменем поля. Усі помилки мають одну форму { statusCode, code, message, requestId }.
Чому важливо	Без цього сміття з фронтенду доїжджає до бази, а помилки виглядають по-різному в кожному ендпоінті. Фронтенд не може їх нормально показати.
Коли про це думати	З першого ендпоінту. Це найдешевша річ у бекенді і найдорожча, якщо додавати потім.
Коли можна забити	Ніколи. Навіть у пет-проєкті на вечір. ValidationPipe це три рядки.
При якому трафіку	Будь-який. Один користувач із кривим запитом достатньо.
Як зрозуміти, що час настав	У логах 500 замість 400. Фронтенд питає «а що мені показати, коли прийшло ось це?».

Пагінація: offset і cursor

core: posts

Що це	Offset: «дай сторінку 7 по 20». Просто, є загальна кількість, але база щоразу пропускає перші 120 рядків. Cursor: «дай 20 після ось цього запису». База стрибає прямо туди, бо є індекс.
Чому важливо	Offset на мільйоні рядків: сторінка 5000 рахується секундами. Ще й рядки стрибають між сторінками, коли хтось щось додав.
Коли про це думати	Коли проєктуєш список, який ростиме: стрічка, історія, логи. Для таблиці на 200 рядків думати не треба.
Коли можна забити	Списки до кількох тисяч рядків. Адмінки. Усе, де є кнопки «1 2 3 ... 10».
При якому трафіку	Таблиця понад 100 тисяч рядків або нескінченний скрол. Тоді cursor.
Як зрозуміти, що час настав	EXPLAIN показує, що запит читає в 50 разів більше рядків, ніж повертає. Глибокі сторінки повільніші за перші.

Міграції

core: migrations

Що це	Схема бази змінюється файлами: кожна зміна це міграція з up і down. Вони запускаються по черзі при старті. synchronize: true, коли ORM сам змінює таблиці, вимкнено.
Чому важливо	synchronize на продакшні може «вирівняти» схему і видалити колонку з даними. Без міграцій два розробники мають дві різні бази і «у мене працює».
Коли про це думати	Із першої таблиці. Перехід на міграції потім, коли є дані, болючий.
Коли можна забити	Прототип на день, який викинуть. І то краще не звикати.
При якому трафіку	Не про трафік. Про команду понад одну людину і базу з даними, які шкода.
Як зрозуміти, що час настав	Колонка «зникла». У колеги не запускається через відсутню таблицю. Деплой вимагає ручних дій у базі.

Індекси і N+1

core: queries

Що це	Індекс це зміст книги: без нього база гортає всі сторінки (Seq Scan), з ним відкриває потрібну (Index Scan). N+1: взяв 30 постів одним запитом, а потім для кожного окремо сходив за автором. 31 запит замість 2.
Чому важливо	Без індексу запит на 200 тисячах рядків у проєкті займає 23 мс, з індексом 0.2 мс. У 100 разів. N+1 на списку з 1000 елементів це 1001 запит на кожне відкриття сторінки.
Коли про це думати	Індекс: коли пишеш WHERE або ORDER BY по полю, якого ще немає в індексі. N+1: кожного разу, коли в циклі є await.
Коли можна забити	Таблиці до 10 тисяч рядків: Seq Scan там швидший за індекс. Не індексуй усе підряд, кожен індекс сповільнює INSERT.
При якому трафіку	Понад 50-100 тисяч рядків у таблиці або понад 10 запитів на секунду до неї.
Як зрозуміти, що час настав	p95 ендпоінту росте разом із розміром таблиці. Лічильник SQL на один запит показує десятки. EXPLAIN пише Seq Scan на великій таблиці.

Кеш у Redis	core: queries (stats)
Що це	Дорогий результат (агрегат, звіт) один раз рахуємо і кладемо в Redis на 60 секунд. Наступні запити беруть звідти. Коли дані змінились, кеш стираємо (інвалідація).
Чому важливо	Одна і та сама важка вибірка 100 разів за хвилину це 100 разів навантаження на базу заради однакової відповіді.
Коли про це думати	Коли один і той самий запит повторюється часто, а відповідь змінюється рідко. Головна сторінка, лічильники, довідники.
Коли можна забити	Дані, які мусять бути свіжими до секунди (баланс), або які у кожного користувача свої. Там кеш лише додає багів «бачу старе».
При якому трафіку	Понад 10 однакових читань на секунду одного важкого запиту. Або один запит довший за 200 мс, який викликають часто.
Як зрозуміти, що час настав	База навантажена на 80 %, а в топі запитів один і той самий SELECT. Заголовок X-Cache: HIT у проєкті показує, чи кеш узагалі працює.

Логи, метрики, трейси	core: logging, observability, tracing
Що це	Логи: що сталось з одним запитом (по requestId видно шлях через API і воркер). Метрики: скільки запитів, як швидко, скільки помилок за час (Prometheus, Grafana). Трейси: де саме всередині запиту пішов час (Jaeger).
Чому важливо	Без них «у мене повільно» розбирається навмання. З ними: дашборд показує p95, трейс показує, що 900 мс із 1000 це очікування пулу, лог показує, хто саме.
Коли про це думати	Логи з requestId з першого дня. Метрики перед запуском у продакшн. Трейси, коли запит проходить більше ніж один процес.
Коли можна забити	Пет-проєкт для себе: console.log і достатньо. Але requestId у ньому все одно додай, це одна middleware.
При якому трафіку	Метрики потрібні, коли ти не можеш прочитати всі логи очима. Це приблизно 1 запит на секунду і більше.
Як зрозуміти, що час настав	Ти відкриваєш логи і не можеш сказати, який рядок до якого запиту належить. Питання «а скільки в нас зараз помилок?» не має відповіді за хвилину.

Тести і CI	core: tests, патерни: ci
Що це	Три рівні: unit (сервіс із фейковим репозиторієм, без бази), інтеграційний (репозиторій проти справжньої бази), e2e (запит через HTTP). CI на GitHub проганяє lint, build і всі тести на кожен push і світить червоним або зеленим.
Чому важливо	Без тестів кожна зміна це «начебто працює». Без CI тести є, але їх ніхто не запускає. Паралельні сценарії (20 списань, 8 webhook) узагалі неможливо перевірити руками.
Коли про це думати	Unit на сервіс одразу, це дешево. e2e на критичний шлях (оплата, вхід) перед запуском. CI у перший день репозиторію, це один файл.
Коли можна забити	Тести на геттери і на те, що фреймворк і так гарантує. Сто відсотків покриття заради цифри.
При якому трафіку	Не про трафік. Про ціну помилки: гроші, дані, репутація.
Як зрозуміти, що час настав	Баг, який уже був виправлений, повернувся. Боїшся чіпати старий код. «Перевір, будь ласка, на проді».

Локи: кредити і FOR UPDATE	core: credits
Що це	Коли два запити одночасно хочуть списати з одного балансу, рядок балансу блокується (SELECT ... FOR UPDATE), другий чекає, поки перший закінчить. Плюс CHECK (balance >= 0) у базі як страховка.
Чому важливо	Без лока обидва читають «50», обидва списують 30, у базі 20 замість помилки. З грошима це пряма втрата. У проєкті тест: 20 паралельних списань по 10 із балансу 50, проходять рівно 5.
Коли про це думати	Щойно з'являється операція «прочитати, порахувати, записати» над одним рядком, яку можуть викликати одночасно: баланс, залишок на складі, лічильник місць.
Коли можна забити	Дані, де гонка нешкідлива: лічильник переглядів, «останній раз онлайн». Там достатньо атомарного UPDATE ... SET x = x + 1.
При якому трафіку	Не про трафік: два одночасні кліки одного користувача вже гонка. Подвійний клік по «Оплатити» трапляється на будь-якому трафіку.
Як зрозуміти, що час настав	Баланс пішов у мінус. Продали більше, ніж було на складі. Тест із паралельними запитами падає.

Черга і воркер	core: jobs
Що це	Усе довше за секунду не робиться в запиті. API відповідає 202 і кладе завдання в чергу (BullMQ у Redis). Окремий процес, воркер, бере завдання і робить. Клієнт питає статус або отримує подію.
Чому важливо	HTTP-запит на 30 секунд відвалюється за таймаутом, а якщо процес API перезапустили, робота зникла. У черзі завдання переживає перезапуск і повторюється при помилці.
Коли про це думати	Щойно з'являється операція довша за 1-2 секунди: звіт, обробка файлу, виклик LLM, відправка пошти. Або щось, що мусить статися навіть якщо API помре.
Коли можна забити	Операції до 500 мс. Пет-проєкт без Redis. Тоді просто зроби це в запиті і не ускладнюй.
При якому трафіку	Будь-який, якщо є довгі операції. Один користувач із 30-секундним звітом уже тримає з'єднання.
Як зрозуміти, що час настав	Таймаути на фронтенді. Запити довші за 5 секунд у логах. Користувачі тиснуть «ще раз» і створюють дублі.

Backpressure і пачки	core: jobs
Що це	Якщо в черзі чекає понад 100 завдань, API відповідає 429 QUEUE_FULL замість того, щоб брати ще. Воркер пише події в базу пачками: один INSERT на 100 рядків замість 100 окремих. У проєкті 459 мс проти 10 мс.
Чому важливо	Черга без межі росте, поки не закінчиться пам'ять Redis. Сто окремих INSERT це сто походів до бази заради того, що можна зробити одним.
Коли про це думати	Коли воркер може не встигати за API: імпорт, події з фронтенду, логи. Пачки: коли записів понад кілька десятків на секунду.
Коли можна забити	Черга, у яку кладуть десять завдань на день. Записи по одному на хвилину.
При якому трафіку	Понад 50 записів на секунду в одну таблицю. Черга, яка в пік росте швидше, ніж спадає.
Як зрозуміти, що час настав	Графік waiting у черзі росте і не повертається до нуля. Redis їсть пам'ять. Воркер завжди зайнятий.

Polling, SSE, WebSocket	core: live
Що це	Три способи дізнатись, що на сервері щось змінилось. Polling: клієнт питає щосекунди. SSE: сервер шле події в один бік по відкритому з'єднанню. WebSocket: відкрита лінія в обидва боки. У проєкті всі три на одному звіті.
Чому важливо	Polling на 1000 клієнтів щосекунди це 1000 запитів на секунду заради «ще не готово». WebSocket без Redis-адаптера на двох процесах доставляє подію половині вкладок.
Коли про це думати	Коли на сторінці є щось, що змінюється без дії користувача: прогрес, чат, ціни, «хто онлайн». Для одного прогрес-бару починай із polling.
Коли можна забити	Дані, які змінюються раз на хвилину. Polling раз на 10 секунд і не треба нічого складного.
При якому трафіку	Понад 100 одночасних клієнтів, які чекають на зміну. Або оновлення частіше за раз на секунду.
Як зрозуміти, що час настав	У логах тисячі однакових GET /status з 200 і порожньою зміною. Клієнти скаржаться, що оновлення приходить із запізненням.

Пул з'єднань до бази	core: pool
Що це	База тримає обмежену кількість з'єднань, застосунок бере їх із пулу (у нас 10). Запит бере з'єднання на мить і повертає. Якщо всі 10 зайняті, одинадцятий чекає.
Чому важливо	Один повільний запит або HTTP-виклик усередині транзакції тримає з'єднання секунди. Десять таких, і весь застосунок стоїть, навіть для простих запитів.
Коли про це думати	Коли пишеш транзакцію: всередині тільки робота з базою, жодних fetch. І коли додаєш другий процес (воркер): у нього свій пул, база має витримати суму.
Коли можна забити	Поки один процес і пул 10 без довгих транзакцій. Налаштування за замовчуванням працюють.
При якому трафіку	Понад 50 запитів на секунду до бази або кілька процесів на одну базу. Тоді рахувати: процеси × pool.max має бути менше за max_connections.
Як зрозуміти, що час настав	У метриках pg_pool_waiting більше нуля довше кількох секунд. Помилка «timeout acquiring connection». Прості ендпоінти раптом повільні разом із важкими.

Rate limit	core: ratelimit
Що це	Кожному користувачу або IP дається відро на 10 запитів за 10 секунд. Запит бере токен, відро повільно наповнюється. Порожнє відро дає 429 і заголовок Retry-After.
Чому важливо	Один скрипт або один баг на фронтенді в циклі кладе весь API для всіх. Ліміт робить так, що страждає тільки винний.
Коли про це думати	Перед публічним запуском. І на всьому, що дороге: логін (перебір паролів), відправка пошти, виклики LLM.
Коли можна забити	Внутрішній сервіс, куди ходить тільки твій фронтенд за VPN. Поки немає жодного зовнішнього клієнта.
При якому трафіку	Публічний API з першого дня. Інакше понад 100 запитів на секунду від одного джерела у логах.
Як зрозуміти, що час настав	Один IP або користувач дає понад половину всіх запитів. Хвилі 500 збігаються з активністю одного клієнта.

Чуже API і його ліміти	core: github
Що це	GitHub дозволяє 100 одночасних запитів і 900 «пунктів» за хвилину. Клієнт у проєкті тримає семафор на одночасність, рахує бюджет і при 429 чекає стільки, скільки сказано в Retry-After, потім повторює.
Чому важливо	Без цього імпорт на 500 репозиторіїв забанить токен на годину, і впаде все, що його використовує, а не тільки імпорт.
Коли про це думати	Перед першим викликом чужого API подивись його сторінку про ліміти. Вона є в усіх: GitHub, Stripe, OpenAI, Telegram.
Коли можна забити	Один-два виклики на хвилину до API з лімітом у тисячі. Досить таймауту і одного повтору.
При якому трафіку	Пакетні операції: імпорт, синхронізація, розсилка. Усе, де в циклі викликається чуже API.
Як зрозуміти, що час настав	У логах 429 або 403 від чужого сервісу. Імпорт «іноді» падає посередині.

Деплой	core: docs/deploy.md
Що це	Той самий docker compose, що й локально, на сервері. CI після зеленого main заходить по SSH, робить git pull і compose up. HTTPS дає nginx з Let's Encrypt.
Чому важливо	Ручний деплой «зайшов, скопіював, перезапустив» ламається на третій раз і не відтворюється. Без HTTPS браузер і так усе заблокує.
Коли про це думати	Коли проєкт побачить хтось, крім тебе. Перший деплой краще зробити рано, поки проєкт маленький.
Коли можна забити	Локальний інструмент тільки для себе.
При якому трафіку	Один безкоштовний VPS або EC2 тягне такий проєкт до сотень запитів на секунду. Далі вже окрема розмова.
Як зрозуміти, що час настав	Ти робиш деплой руками частіше за раз на тиждень. Після деплою «щось не те», а що змінилось, незрозуміло.

4. patterns: реальні кейси

Порядок теж навчальний: перші шість кейсів живуть усередині одного запиту, з нотифікацій з'являються воркер і події. Майже всюди в полі «При якому трафіку» написано «не про трафік». Це не відмазка: ці проблеми приходять не від навантаження, а від того, що світ не ідеальний: мережа повторює, процеси падають, люди клацають двічі.

Ідемпотентність: webhook двічі	patterns: idempotency
Що це	Платіжка шле «оплата пройшла» і може прислати це двічі або двома паралельними запитами. Id події записується в таблицю з первинним ключем у тій самій транзакції, що й оновлення замовлення. Другий запис падає на ключі, і замовлення лишається оплаченим один раз.
Чому важливо	Інакше один платіж двічі зараховується, двічі шле товар, двічі списує кредити. Stripe, Monobank, LiqPay прямо пишуть у документації: ми можемо прислати повторно.
Коли про це думати	Щойно інтегруєш будь-який webhook або робиш POST, який не можна виконати двічі (оплата, переказ, створення замовлення).
Коли можна забити	GET-запити. POST, повтор якого нешкідливий (лайк, який і так унікальний по індексу).
При якому трафіку	Не про трафік. Повтор трапляється через мережу, а не через навантаження.
Як зрозуміти, що час настав	У базі два платежі з одним id події. Користувач отримав два листи «дякуємо за замовлення».

Оптимістичне блокування	patterns: optimistic-lock
Що це	У рядку є поле version. Клієнт при редагуванні надсилає version, який бачив. UPDATE ... WHERE version = 7. Якщо хтось уже змінив, рядків оновлено 0, і клієнт отримує 409 з актуальною версією.
Чому важливо	Два менеджери відкрили картку товару. Перший зберіг ціну, другий через хвилину зберіг опис і затер ціну старим значенням. Ніхто не помітив.
Коли про це думати	Форми редагування, які можуть відкрити двоє одночасно. Усе, що люди правлять руками: товари, статті, налаштування.
Коли можна забити	Дані, які змінює один користувач (свій профіль). Поля, які безпечно затерти.
При якому трафіку	Не про трафік. Про кількість людей, які правлять одне й те саме.
Як зрозуміти, що час настав	«Я ж змінював, куди поділось?». Історія змін показує, що значення стрибає туди-сюди.

Soft delete і аудит

patterns: audit

Що це	Видалення ставить deletedAt замість того, щоб стерти рядок. Є restore. Кожна зміна товару або замовлення пише рядок в audit_log: що було, що стало, хто, коли, з яким requestId.
Чому важливо	«Я випадково видалив», і даних немає. «Хто поміняв ціну на 1 гривню?», і ніхто не знає.
Коли про це думати	Для всього, що має цінність для бізнесу: товари, замовлення, користувачі, гроші. І для всього, що редагують люди.
Коли можна забити	Технічні таблиці: сесії, кеш, логи. Там видаляти по-справжньому.
При якому трафіку	Не про трафік. Про ціну втрати і про те, чи питатимуть «хто».
Як зрозуміти, що час настав	Перший запит «поверни, будь ласка» або «хто це зробив». Якщо проєкт живий, він буде.

Multi-tenant

patterns: multi-tenant

Що це	Один застосунок обслуговує кілька магазинів. У кожній таблиці є tenant_id, береться з JWT, репозиторій сам додає WHERE tenant_id = ... до всього. Чужий id дає 404, а не 403, щоб не видати, що запис існує.
Чому важливо	Один пропущений WHERE, і магазин А бачить замовлення магазину В. Це не баг, це витік даних і кінець довіри.
Коли про це думати	З першої таблиці, якщо клієнтів буде більше одного. Додати tenant_id потім до 40 таблиць із даними це тиждень болю.
Коли можна забити	Проєкт для одного клієнта і такого плану немає.
При якому трафіку	Не про трафік. Про кількість клієнтів із окремими даними.
Як зрозуміти, що час настав	Другий клієнт. Або перше «а можна нам окремий акаунт для філії».

Feature flags і версії API

patterns: feature-flags-and-versioning

Що це	Прапорець у базі вмикає функцію для 10 % користувачів, потім для всіх, без деплою. /v1/products і /v2/products живуть поруч, старий фронтенд працює, новий переходить, коли готовий.
Чому важливо	Зміна форми відповіді ламає всіх, хто не оновився: мобільний застосунок у сторі оновлюється тижнями. Відкат функції через деплой це 20 хвилин, через прапорець 2 секунди.
Коли про це думати	Коли є клієнт, який оновлюється не разом із бекендом: мобільний застосунок, чужа інтеграція. Прапорці: коли боїшся вмикати на всіх одразу.
Коли можна забити	Фронтенд і бекенд деплояться разом і клієнт один. Тоді просто міняй.
При якому трафіку	Не про трафік. Про кількість незалежних клієнтів API.
Як зрозуміти, що час настав	Після релізу частина користувачів бачить помилки, бо в них стара версія. Хочеться «спочатку на 5 %».

Файли через presigned URL

patterns: files

Що це	Браузер просить у API дозвіл, отримує підписане посилання і вантажить файл прямо в S3 або MinIO. Байти не проходять через API. Потім API перевіряє, що файл є, і записує ключ.
Чому важливо	Через API 100 людей вантажать по 5 МБ, і він зайнятий тільки цим. Пам'ять, диск, з'єднання, усе на файли замість логіки.
Коли про це думати	Коли файли більші за кілька сотень кілобайт або їх більше ніж кілька на хвилину.
Коли можна забити	Аватарки 50 КБ раз на день. Можна через API, зберігаючи в S3.
При якому трафіку	Понад кілька мегабайт на секунду сумарно. Або будь-які відео.
Як зрозуміти, що час настав	API-процес «важкий» у метриках пам'яті саме в моменти завантажень. Таймаути на великих файлах.

Нотифікації з dead-letter чергою

patterns: notifications

Що це	«Лист про оплату» це завдання в черзі. Три спроби з паузою, потім завдання переїжджає в окрему чергу «dead-letter», де його видно і можна повторити руками. Bull Board показує обидві.
Чому важливо	Лист, який не пішов, бо SMTP тимчасово впав, інакше зникне назавжди. Або навпаки, буде повторюватись вічно і забивати чергу.
Коли про це думати	Будь-яка дія «назовні» після події: пошта, SMS, push, webhook клієнту.
Коли можна забити	Нотифікації, втрата яких нічого не значить (ефемерний toast в інтерфейсі).
При якому трафіку	Не про трафік. Про наслідок втрати.
Як зрозуміти, що час настав	«Я не отримав листа», і неможливо сказати чому. Черга забита одним завданням, яке падає щосекунди.

Outbox

patterns: outbox

Що це	Треба зберегти замовлення і повідомити інші частини системи. Замість «зберіг, потім відправив» подія пишеться в таблицю outbox у тій самій транзакції. Окремий relay читає таблицю і публікує в чергу.
Чому важливо	Між «зберіг» і «відправив» процес може впасти. Замовлення є, події немає, лист не пішов, склад не оновився. І ніхто не дізнається.
Коли про це думати	Коли одна дія має викликати іншу в іншому процесі або сервісі, і втрата недопустима: оплата, замовлення, реєстрація.
Коли можна забити	Усе в одному процесі і події нікуди не йдуть. Події, втрата яких нічого не ламає (аналітика).
При якому трафіку	Не про трафік. Про наслідки втрати однієї події.
Як зрозуміти, що час настав	«Замовлення є, а лист не прийшов» хоча б раз. Розбіжності між таблицями, які мають збігатись.

Saga: кілька кроків з відкатом

patterns: saga

Що це	Checkout це три кроки: зарезервувати склад, списати оплату, створити доставку. Якщо другий упав, перший треба відкотити. Стан saga зберігається в таблиці, тому після падіння процесу воркер продовжить із того місця.
Чому важливо	Без цього зависають «напівзамовлення»: склад зарезервовано, оплата не пройшла, резерв ніхто не повернув. Через тиждень товару «немає», хоча він на полиці.
Коли про це думати	Коли один сценарій зачіпає два і більше незалежних ресурси (своя база плюс чуже API), і їх не можна накрити однією транзакцією.
Коли можна забити	Усі кроки в одній базі: тоді це звичайна транзакція, saga не потрібна.
При якому трафіку	Не про трафік. Про кількість зовнішніх систем у одному сценарії.
Як зрозуміти, що час настав	Резерви, які ніколи не звільняються. Оплати без замовлення. Ручні «підчистки» бази раз на тиждень.

Circuit breaker

patterns: circuit-breaker

Що це	Чуже API лягло. Замість того, щоб кожен запит чекав таймаут 2 секунди, після 5 збоїв за 10 секунд «запобіжник» розмикається: запити одразу отримують 503 за 5 мс. Через 30 секунд один пробний запит перевіряє, чи ожило.
Чому важливо	Без цього кожен запит до мертвого сервісу тримає з'єднання і воркер 2 секунди. Сто запитів, і твій застосунок теж мертвий, хоча зламався не він.
Коли про це думати	Для кожного зовнішнього виклику на критичному шляху: платіжка, SMS, геокодер.
Коли можна забити	Виклик, який і так робиться у фоні з повторами (черга). Виклик до власної бази.
При якому трафіку	Понад кілька запитів на секунду до одного зовнішнього сервісу.
Як зрозуміти, що час настав	Коли лягає чужий сервіс, лягає і твій. Трейси показують, що весь час запиту це очікування чужої відповіді.

Профілювання

patterns: profiling

Що це	Звіт працює 10 секунд. Замість гадати: console.time по секціях, лічильник SQL-запитів (у проєкті показав 20 030 на один звіт), EXPLAIN, у крайньому разі node --cpu-prof. Після виправлення N+1: 50 мс і 1 запит.
Чому важливо	Без вимірювання оптимізують не те: додають індекс, а проблема в N+1. Дві години на те, що видно за дві хвилини.
Коли про це думати	Коли щось повільне і ти не знаєш чому. Тобто завжди перед тим, як щось «пришвидшувати».
Коли можна забити	Коли і так ясно (один запит без індексу на мільйоні рядків). Тоді просто EXPLAIN.
При якому трафіку	Не про трафік.
Як зрозуміти, що час настав	Ти кажеш «здається, повільно через ...». Слово «здається» і є сигнал.

5. Де дивитись на цифри

Картки раз у раз кажуть «подивись у метрики» або «подивись у логи». Ось що для цього є в проєктах і де воно живе.

Логи	JSON-рядки з requestId у кожному запиті і в кожному завданні воркера. У Grafana через Loki: вписав requestId, бачиш усе про один запит.	core: logging
Метрики	GET /metrics у API і воркері (Prometheus). Дашборд Grafana: запитів за секунду, p50/p95, помилки, пул, черги, пам'ять.	core: observability
Трейси	Jaeger: один запит розкладений по крокам із часом кожного, включно з воркером.	core і patterns: tracing
Черги	Bull Board на /queues: скільки чекає, в роботі, впало, повторюється.	core, patterns
База	EXPLAIN на вимогу (/demo/explain), лічильник SQL на запит (/demo/n-plus-one).	core: queries

Запуск усього разом: `docker compose --profile observability up --build` у backend-2-core. Grafana на localhost:3001, Prometheus на 9090, Jaeger на 16686. Детально: docs/observability.md і docs/tracing.md.

Сценарій, який варто пройти один раз

- Відкрити Grafana, знайти ендпоінт із найбільшим p95.
- Відкрити Jaeger, знайти трейс цього ендпоінту, побачити, який крок з'їв час (пул? база? чужий API?).
- Скопіювати requestId із трейсу, знайти в Loki всі рядки логів цього запиту.
- Після цього слова «повільно» і «щось впало» перестають бути загадкою.

Що перевірено, а що ні

- Усі тести трьох проєктів прогнані на справжніх PostgreSQL 16 і Redis 7. API і воркери запускались як окремі процеси, смоук-скрипти пройшли, сторінки learn прокликані Playwright на живому бекенді.
- Не запускалось у середовищі збірки (там немає Docker): docker compose, nginx, MinIO, Jaeger, Grafana, Prometheus, Loki, pgvector. У compose усе описано, образи стандартні, але перший `docker compose up --build` треба зробити локально і подивитись.
- Справжні GitHub API, OpenAI і AWS S3 не викликались: скрізь є фейкові адаптери, справжні вмикаються змінними в .env.

AWS

Акаунт створюється самостійно. Поки він не підтверджений, усе працює локально: MinIO замість S3, деплой на будь-який VPS. Після підтвердження чотири змінні в .env переключають файли на S3, а docs/deploy.md у core описує деплой на EC2.